

Homework 2: ARC Search ↕

- Due 2 Oct by 23:59
- Points 100
- Submitting an external tool

Problem Solving: ARC Prize Planning

Introduction:

While each puzzle in the ARC prize is unique, certain concepts, such as rotation and mirroring options, occur repeatedly. One can think of these operations as a set of building blocks that encapsulate those concepts and can then be used to build a solution program, that is, task-specific programs that correctly transform each of the input grids of a given task into its corresponding output grid. Such a set of building blocks adds up to form a domain-specific language (DSL).

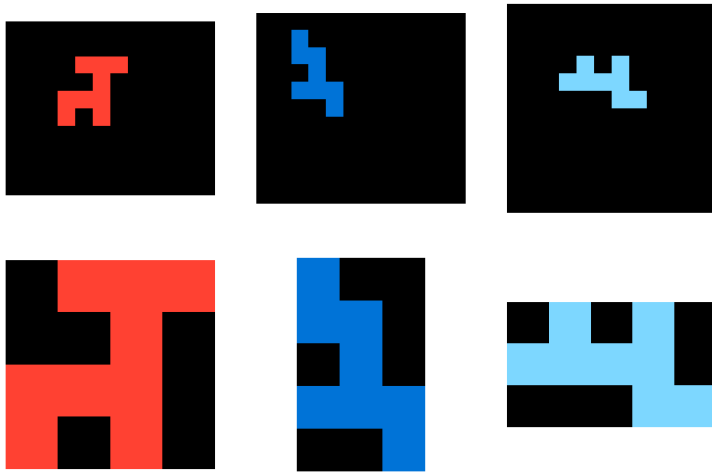
Your goal is to implement a solution that plans and sequences such transformations to achieve the desired output from a given input. The ARC problem can be approached using planning techniques commonly used in AI. These techniques break down complex tasks into smaller, more manageable steps using a series of primitive operations. Your task is to use such operations to solve a sequence of transformations, effectively planning a solution.

Problem Description:

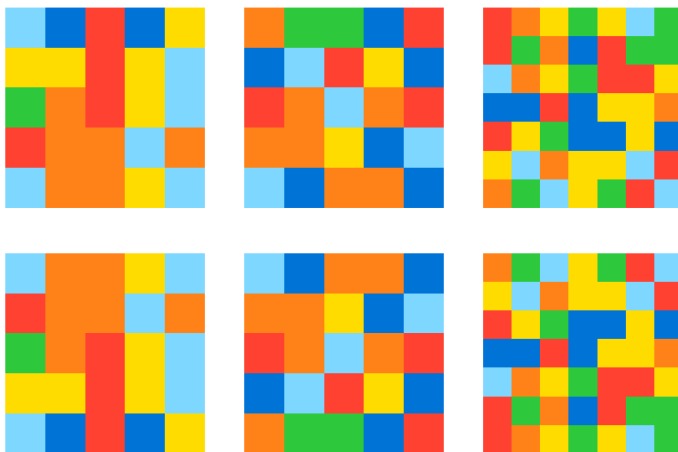
Given a starting grid (input frame), your job is to figure out the set of steps (primitive operations) that will transform it into the target grid (final frame). You will develop a function that automatically determines the appropriate sequence of actions needed to achieve this transformation. You will apply various search algorithms to solve the ARC puzzle transformations. Some of these algorithms include **depth-first search**, **breadth-first search**, and **best-first search**. These methods will explore different sequences of actions to transform an input grid into the target output grid. To guide the search, your solution may include a heuristic function that estimates the distance from the current state to the goal state. Heuristic-based searches, such as A*, use this information to explore promising paths more efficiently.

Primitive Operations: Below are examples of primitive operations within the ARC domain.

(i) *Compression*: This operation removes all rows or columns that contain a single value. This often has the effect of removing the space around objects (i.e., it removes the "frontiers").

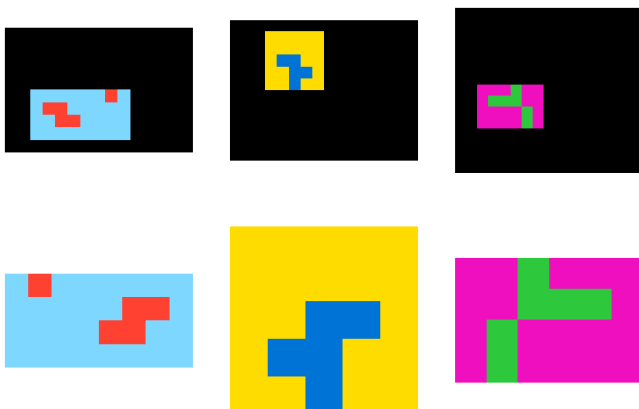


(ii) *Horizontal Mirroring*: This operation mirrors the grid along the horizontal axis.

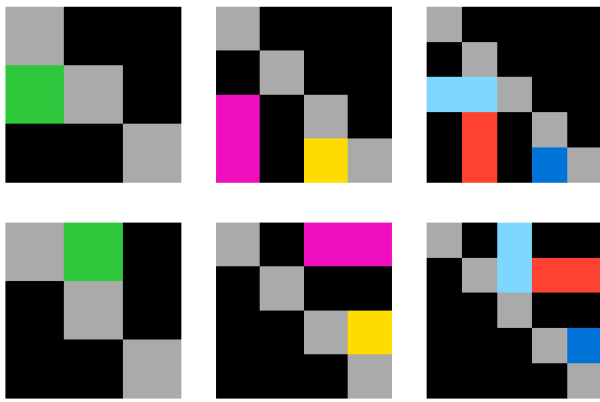


(iii) *Sequence of Actions*: You can combine multiple primitive operations into a sequence. For example:

- Rotate by 90, rotate by 90, mirror horizontally, and then compress.



- Mirror horizontally, rotate by 90 degrees, and then compress.



Task:

Your goal is to implement a function called `arc_planning` that can determine the correct sequence of actions to apply to an input grid, resulting in the target output grid. You will use the search or planning methods covered in class to develop an efficient solution.

Starter Code:

We provide you with [starter code \(https://gatech.instructure.com/courses/534944/files/76043603?wrap=1\)](https://gatech.instructure.com/courses/534944/files/76043603?wrap=1) [↓ \(https://gatech.instructure.com/courses/534944/files/76043603/download?download_frd=1\)](https://gatech.instructure.com/courses/534944/files/76043603/download?download_frd=1) that includes an implementation of the `ArcPuzzle` class and search algorithms to help you focus on developing the planning logic. Be sure to inspect the actions that this class supports. Your main task will be to implement the `arc_planning` function using the provided framework.

The `arc_planning` function will take the initial grid and transform it into the target grid using a sequence of these operations. The function should return a list of moves (operation names) that transform the input grid into the output grid. For example:

```
return ["rot90", "hmirror", "compress"]
```

You are expected to use various search algorithms, such as breath-first search or depth-first search, to explore possible sequences of actions and return a sequence that transforms the input grid into the output grid.

Submitting Your Solution

To submit your code, go to the course in Canvas and click **Gradescope** on the left side. Then, select **CS 4635/7637** if needed. You will see an assignment named Homework 2. Select this project, then drag your `arc_puzzles.py` file into the autograder. Please put your entire implementation in this one file. If you submit other files or rename the file, the autograder may not run correctly.

How You Will Be Graded

Your solution will be tested using a set of initial and goal state pairs, with different levels of complexity. These tests will evaluate your program's ability to find the correct transformation

sequence for puzzles of increasing difficulty. You are encouraged to attempt solving all the test cases.

Every time you submit a solution to gradescope, we randomly select 16 ARC states from the public training set, then apply anywhere from 1-5 random actions to transform it into a goal state. We then evaluate your submission and our reference submission against this random ARC Puzzle and compare the results. Given that we do not use fixed test cases, we don't have a set of tests we can easily share. However, you are welcome to use the `.randomize(num_shuffles)` method to generate your own train/test cases.

Libraries Allowed

You do not need to make use of any external libraries for this project (none will be available on the autograder instance). The [py_search](https://github.com/cmaclell/py_search) and [py_plan](https://github.com/cmaclell/py_plan) libraries that were discussed in class will be installed in the autograder and available to use. Additionally, we use Python 3 for our autograder.

Inspiration: [Kaggle](https://www.kaggle.com/code/michaelhodel/program-synthesis-starter-notebook) notebook.

Submit Programming Assignment

 Upload all files for your submission

Submission Method

 Upload

 GitHub

 Bitbucket

Drag & Drop

Any file(s) including .zip. Click to browse.